



FINTRACKER

Procesamiento & Análisis de los Datos

Mateo Pérez Suarez (79439806Z)
Iñigo Peña de las Heras (20983346D)

Repositorio del Proyecto

<https://github.com/MateoPerezSuarez/FinTracker>

Los notebooks se encuentran en el directorio `src/Representacion_del_lenguaje`.

29 de octubre de 2025

Índice

1. Extracción y procesamiento inicial de los Datos	2
1.1. Extracción desde la API (finnhub.py)	2
1.2. Resolución de URLs originales (modificarURLs.py)	2
1.3. Scraping de artículos (scraperTextos.py)	2
1.4. Proceso de datos no labelados	3
2. Preprocesado de los datos	4
2.1. No contextuales	4
2.1.1. Selección de herramientas y procesos	4
2.1.2. Implementación del Preprocesamiento	5
2.1.3. Análisis de Resultados	6
2.2. Contextuales	7
2.2.1. Tokenización para BERT	7
2.2.2. Tokenización Fin-BERT	7
3. Embeddings	8
3.1. Métodos de representación tradicionales	8
3.1.1. Análisis comparativo de resultados	8
3.2. No contextuales	10
3.2.1. Modelos implementados	10
3.2.2. Cobertura y calidad de los embeddings	10
3.2.3. Análisis semántico y visualización	11
3.2.4. Conclusiones y modelo más adecuado	12
3.3. Contextuales	12
3.3.1. Embeddings BERT	12
3.3.2. Embeddings Fin-Bert	13
4. Resumen de Embeddings y Aplicaciones por Tarea	13

1 Extracción y procesamiento inicial de los Datos

Antes de detallar cada uno de los procesos, cabe mencionar que toda esta parte del pipeline se encuentra implementada dentro de la ruta `src/dataProcessing/` del proyecto. En esta carpeta hemos incluido los módulos principales empleados para la recopilación y preparación de los datos:

- `finnhub.py`: extracción de noticias mediante la API oficial de Finnhub.
- `modificarURLs.py`: resolución de las URLs originales de los artículos.
- `scrapperTextos.py`: scraping y limpieza del texto principal de las noticias.

Cada script cumple una función específica dentro del flujo de generación del dataset final que usaremos para alimentar los modelos en un futuro.

1.1 Extracción desde la API (`finnhub.py`)

En primer lugar, para la extracción de datos desde la API mediante peticiones, se descargaron noticias de cada empresa en intervalos de siete días hasta recopilar aproximadamente 1000 noticias de cada una de las principales compañías (Apple, Amazon, Google, Meta, Microsoft, Nvidia y Tesla). Además, se recopilaron noticias de otras empresas de menor tamaño con el objetivo de enriquecer el dataset y evaluar la capacidad de generalización de los modelos frente a compañías no vistas durante el entrenamiento.

Cada respuesta en formato **JSON** recibida desde la API se almacenó en un dataset distinto, separado por empresa en el caso de las grandes y en un único dataset conjunto para las pequeñas. Finnhub proporciona de forma nativa campos como el *headline* (titular), *summary* (resumen) y *related* (empresa mencionada en la noticia). Por tanto, uno de los pasos siguientes consistió en procesar estos datos no etiquetados, generando las *topics* de las noticias y extrayendo el texto completo de cada artículo.

1.2 Resolución de URLs originales (`modificarURLs.py`)

Las URLs proporcionadas por Finnhub suelen ser redirecciones que no apuntan directamente al artículo original. Para solventar esto, se desarrolló un sistema que sigue los enlaces hasta obtener la URL final del artículo. Se emplearon las librerías **requests** y **BeautifulSoup**, identificando etiquetas HTML relevantes como `<link rel=canonical>` o `<meta property=.og:url>` para recuperar la dirección definitiva del contenido.

1.3 Scraping de artículos (`scrapperTextos.py`)

Una vez resueltas las URLs originales, se utilizó la librería **Trafilatura** para realizar el proceso de scraping y extraer únicamente el texto principal del artículo, que fue almacenado en la columna `article_text` del dataset. Durante este proceso se detectaron

numerosos textos inválidos, como mensajes de error o avisos sobre cookies. Para mejorar la calidad del corpus, se filtraron los artículos con menos de 100 caracteres o que contenían frases recurrentes de error (por ejemplo, “Oops, something went wrong” o “Terms and privacy policy”).Barbaresi (2021)

1.4 Proceso de datos no labelados

En cuanto a la task principal, dado que nuestro dataset carecía de etiquetas sobre el tipo de noticia financiera, hemos tenido que generarlas manualmente. Para ello, hemos utilizado la librería *TopicWizard*, para agrupar los textos del corpus en conjuntos temáticamente similares según la frecuencia y coincidencia de palabras. A partir de estos grupos, identificamos las palabras más representativas y se asignó a cada conjunto un nombre de “*topic*”, con el fin de clasificar e interpretar los textos de forma semántica.

Para realizar la agrupación de textos, aplicamos un proceso de vectorización y descomposición semántica. Primero, cada documento fue tokenizado y lematizado mediante una función personalizada que elimina los caracteres no alfabéticos, aplica case folding y reduce las palabras a su forma base con *WordNetLemmatizer*.

Posteriormente, utilizamos un *CountVectorizer* como implementación de Bag of Words para transformar el corpus en una matriz de frecuencias de términos, representando así cada texto como un vector numérico. A esta matriz le aplicamos un modelo de *Non-negative Matrix Factorization (NMF)* con 5 componentes, encargado de descomponer el espacio de palabras en un conjunto de temas latentes.

El pipeline completo (vectorización y modelado) lo hemos implementado mediante la librería *TopicWizard*, que además permitió visualizar los clusters temáticos e identificar las palabras más representativas de cada grupo, con las que interpretamos cada grupo y les pusimos los siguientes nombre de “topic”:

1. Business Growth and Cloud Infrastructure in the AI Industry:

Las palabras clave hacen referencia a empresas tecnológicas, IA, crecimiento y cloud computing, reflejando la expansión del sector.

2. Financial and Market News and Corporate Sales:

Agrupar textos sobre resultados económicos, cotizaciones y ventas de grandes compañías, especialmente del ámbito financiero.

3. Stock Market and Trading:

Incluye términos asociados al trading, inversión, ETFs y criptomonedas, por lo que describe la actividad de los mercados financieros.

4. Quantum Computing and Military Technology:

Reúne vocabulario técnico sobre computación cuántica, industria y defensa, indicando un tema centrado en innovación tecnológica y militar.

5. Informal / Conversational Lenguaje:

Contiene expresiones coloquiales y no temáticas, propias de citas o lenguaje informal, sin coherencia semántica relevante.

2 Preprocesado de los datos

2.1 No contextuales

En esta fase hemos centrado nuestro enfoque en construir un corpus homogéneo y representativo del dominio financiero, de modo que las señales semánticas (como términos económicos o referencias a entidades) no quedaran diluidas entre ruido textual o variaciones superficiales. El objetivo era preparar los textos para la generación de embeddings no contextuales (Bag-of-Words, TF-IDF y Word2Vec) asegurando que la información relevante se mantuviera de forma consistente.

Para ello, buscamos principalmente dos cosas: por un lado, unificar las formas léxicas y normalizar la escritura para que distintas variantes morfológicas o tipográficas de una misma palabra contribuyeran a una representación única; y por otro, reducir la dimensionalidad y la dispersión del vocabulario, eliminando elementos redundantes o poco informativos.

2.1.1. Selección de herramientas y procesos

Para tokenización, normalización y lematización hemos optado por usar la librería spaCy en lugar de NLTK. Esta decisión viene dada porque encontramos un modelo de lenguaje preentrenado, `en_core_web_sm`, el cual nos agruparía múltiples tareas lingüísticas en un único pipeline optimizado, que abarca tokenización, POS-tagging y lematización de manera cohesiva.

Al cargar el modelo de spaCy, desactivamos explícitamente el parser, textcat y NER porque es innecesario para obtener los lemmas y re-etiquetaría nuestros tokens normalizados creados manualmente cosa que no nos interesa.

En cuanto a la elección entre lematización y stemming, nos decantamos por la primera por razones tanto técnicas como prácticas. La lematización preserva formas canónicas que existen realmente en el diccionario, lo que mejoraría significativamente la interpretabilidad de nuestro vocabulario resultante y evita la generación de raíces artificiales que carecen de significado lingüístico. Además, los modelos preentrenados de spaCy tienen una mayor precisión en la conservación del significado semántico durante el proceso de reducción morfológica, un aspecto muy importante para este tipo de embeddings.

2.1.2. Implementación del Preprocesamiento

Hemos ejecutado el preprocesamiento a través de un pipeline de dos fases, por que queríamos definir una mejor limpieza y marcar bien los pasos:

La primera de las fases se enfoca en manejar la limpieza inicial del texto y la normalización mediante la aplicación de reglas (función `pre_rules` en el código) con distintas finalidades. En primer lugar, decidimos limpiar ruido estructural, eliminando elementos como URLs, etiquetas de HTML y textos boilerplate (read more.º copyright") que han podido surgir a la hora de hacer el scraping, además de limpiar la puntuación tipográfica y los caracteres residuales. En segundo lugar, aplicamos **Case Folding**, convirtiendo todo el texto a minúsculas para gestionar la unificación de tokens (por ejemplo, evitando que el modelo distinga entre "Down" al inicio de una frase y "down" como adjetivo).

El tercer componente de esta fase, es la implementación de un sistema de **placeholders** semánticos. Durante el análisis exploratorio del corpus de noticias financieras, identificamos que la abundante presencia de cifras numéricas, porcentajes y cantidades monetarias con valores específicos altamente variables incrementaba dramáticamente la sparsity del vocabulario. Para mitigar este problema, desarrollamos un sistema que reemplaza estas entidades por tokens genéricos que representan su categoría semántica. Como por ejemplo:

- 10.5 % o 20 per cent → `__PERCENT__`
- \$10 million o €20bn → `__MONEY__`
- 2024, 1998 → `__YEAR__`
- \$TSLA → `__TICKER__`

En cuanto a la segunda fase (función `spacy_clean_strong`), es cuando utilizamos el pipeline de spaCy para la lematización y el filtrado selectivo. Aplicamos la lematización para obtener la forma base de la palabra y se eliminan las stopwords estándar, ya que en modelos no contextuales eliminan el ruido estadístico más frecuente, reduciendo la dimensionalidad. Sin embargo, hemos tratado de llevar acabo una mejor gestión de acrónimos y términos clave. Creamos una lista **KEEP_TERMS** con términos cruciales del dominio (como ONS, UK, GDP) y patrones mediante expresiones regulares (`ACRONYM_RE`), para evitar que estos sean eliminados o mal lematizados por error cosa que nos pasó en un principio. Con esto hemos decidido que se mantienen los tokens alfabéticos de una longitud mínima, asegurando que la representación final se centre exclusivamente en el contenido semántico.

2.1.3. Análisis de Resultados

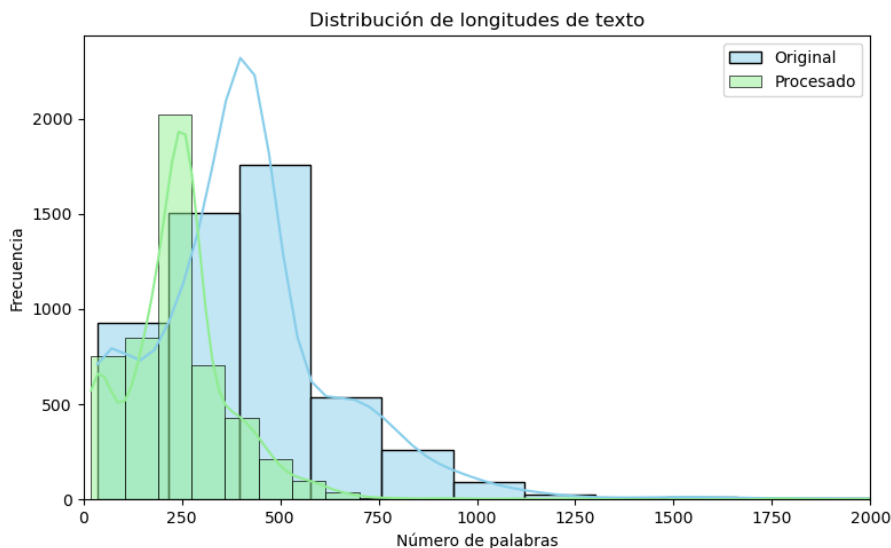


Figura 1: Distribución de longitudes de texto antes y después del preprocesamiento.

En la Figura 1 se aprecia una clara reducción en la longitud de los textos tras aplicar el preprocesamiento. La media de palabras por noticia pasa de 426.9 a 253.6, mientras que la mediana disminuye de 404 a 245, lo que supone una reducción media del 38.9%. La forma de la distribución se mantiene, aunque desplazada hacia valores menores, lo que nos confirma que en el proceso eliminamos principalmente ruido y redundancia sin alterar la estructura ni el contenido semántico principal de los textos.

ANÁLISIS TOKEN POR TOKEN (primeros 60 tokens):				
TOKEN	LEMMA	POS	¿MANTENER?	RAZÓN
the	the	DET	NO	Stopword
uk	uk	PROPN	SÍ	KEEP_TERMS
jobs	job	NOUN	SÍ	Palabra válida
market	market	NOUN	SÍ	Palabra válida
continues	continue	VERB	SÍ	Palabra válida
to	to	PART	NO	Stopword
show	show	VERB	NO	Stopword
signs	sign	NOUN	SÍ	Palabra válida
of	of	ADP	NO	Stopword
weakness	weakness	NOUN	SÍ	Palabra válida
,	,	PUNCT	NO	Puntuación/Espacio
with	with	ADP	NO	Stopword
pay	pay	NOUN	SÍ	Palabra válida
growth	growth	NOUN	SÍ	Palabra válida
...				

Figura 2: Ejemplo del análisis token a token: filtrado y decisión de mantenimiento.

El análisis token a token nos da evidencias de que el pipeline diferencia correctamente entre términos informativos y ruido lingüístico. Palabras funcionales como *the*, *to* o *of* se eliminan como stopwords, mientras que términos sustantivos (*market*, *growth*, *weakness*)

se mantienen. Además, el sistema que hemos diseñado de `KEEP_TERMS` funciona y garantiza que terminos relevantes del dominio financiero, como *UK* o *ONS*, evitando su eliminación errónea durante la lematización. En conjunto, hemos logrado un equilibrio adecuado entre limpieza y retención semántica, reduciendo el vocabulario en torno al 40 % sin perder información relevante.

2.2 Contextuales

En la creación de *embeddings contextuales*, cada palabra obtiene un vector distinto según el contexto, por lo que necesitamos los textos lo más natural y completo posible para capturar el contexto. Para cumplir con esto, la fase de pre-procesamiento debe ser simple, enfocándose en la tokenización precisa y la normalización de casos, por lo que evitaremos practicas como la eliminación de stopwords, stemming o lemmatization.

2.2.1. Tokenización para BERT

Selección de Herramientas e Implementación

A diferencia de ELMo, que utiliza la tokenización de palabras tradicional, los modelos de tipo Transformer como BERT utilizan un proceso llamado tokenización de Sub-word (como WordPiece). Este proceso debe realizarse con el tokenizador específico con el que fue entrenado el modelo. En este caso hemos elegido trabajar el modelo '*bert-base-multilingual-cased*', un modelo muy comunmente utilizado y con una alta disponibilidad y respaldo continuo de la comunidad. En el proceso de tokenización aplicamos padding, para que se trabaje con matrices cuadradas y representacion de los textos de la misma longitud. Tambien vamos a limitar la extension de los textos a 512 tokens, a pesar de que al truncar los textos que superan el limite, se pierde información ,y por tanto, parte del contexto. Esto resultaria en un problema si los textos fuesen largos y tuviesen información relevante más allá del token 512, pero para muchas tareas, los fragmentos más relevantes suelen estar al inicio (titular, primeras frases), y en clasificación de noticias, o incluso NER en párrafos cortos, esto suele bastar.

2.2.2. Tokenización Fin-BERT

Selección de Herramientas e Implementación

En la estructura de nuestro proyecto, una de las tareas trata de extraer eventos o terminos financieros del texto, aplicando Named Entity Recognition. Para ello utilizaremos el modelo Boltuix (2024)NeuroBERT que hemos encontrado en HuggingFace. FinBERT fue entrenado específicamente sobre textos del ámbito financiero, utilizando como base el modelo BERT y adaptando su vocabulario y pesos a terminología propia de noticias, informes y análisis económicos. Por ello, el proceso de tokenización en FinBERT es un paso esencial para garantizar la correcta interpretación del lenguaje financiero.

FinBERT emplea el mismo tipo de tokenización por subpalabras (Subword Tokenization) que BERT, implementada mediante el algoritmo WordPiece, pero entrenada sobre un corpus financiero especializado.

Este modelo destaca por identificar 36 tipos de entidades (por ejemplo, personas, lugares, organizaciones, fechas, dinero) en texto en inglés, lo que lo hace ideal para la tarea de extracción de información. De igual manera, este modelo mapea cada token a su ID numérica correspondiente en el vocabulario de BERT, y rellena las secuencias cortas con el token [PAD], limitando las secuencias a 512 tokens.

3 Embeddings

3.1 Métodos de representación tradicionales

Para comenzar el proceso de vectorización de nuestro corpus, hemos aplicado dos enfoques clásicos: **Bag-of-Words (BoW)** y **TF-IDF (Term Frequency–Inverse Document Frequency)**.

En primer lugar, mediante el modelo **Bag-of-Words**, cada documento se ha representado como un vector de frecuencias, contabilizando el número de apariciones de cada término dentro del vocabulario total. Este enfoque, aunque sea más sencillo, nos ha permitido obtener una primera representación del corpus, con la que hemos podido analizar la distribución de términos y el grado de variabilidad léxica entre noticias. Los resultados han reflejado un vocabulario extenso cosa que consideramos que tiene sentido al tratarse de textos financieros procedentes de distintas fuentes, lo que se ha traducido en una alta dispersión de términos poco recurrentes.

Posteriormente, hemos generado la matriz **TF-IDF**, que ha ajustado las frecuencias obtenidas en BoW en función de la relevancia global de cada palabra en el conjunto de documentos. Este método ha reducido el peso de los términos más comunes y ha resaltado aquellos que aportan información más específica sobre cada noticia. La matriz resultante ha presentado unas dimensiones de **(5160 × 104149)** y una **dispersión del 96.5 %**, lo que implica que solo un 3.5 % de las celdas han contenido valores distintos de cero. Este grado de sparsity ha sido habitual en representaciones TF-IDF aplicadas a corpus amplios y diversos, especialmente en contextos financieros donde los textos comparten un vocabulario limitado.

3.1.1. Análisis comparativo de resultados

Mediante un análisis conjunto de ambos métodos hemos visto diferencias claras en la forma en que representan la información. El modelo BoW refleja de manera más directa la frecuencia absoluta de las palabras, por lo que ha tendido a estar dominado por términos muy comunes (*market*, *company*, *stock*, etc.). Por el contrario, TF-IDF ha reducido el

impacto de esas palabras frecuentes, elevando el peso de términos más distintivos como *merger*, *quarterly* o *guidance*, asociados más eventos o reportes financieros concretos.

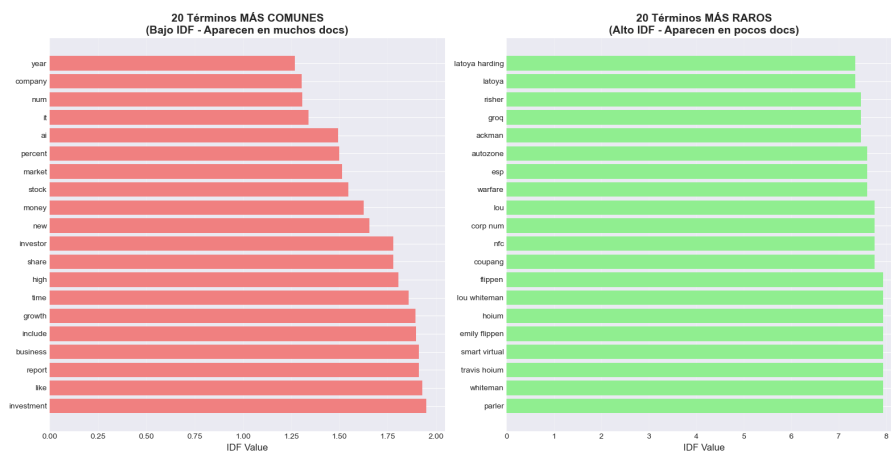


Figura 3: Términos más frecuentes según BoW y TF-IDF.

A partir de la comparación de las matrices de similitud generadas con ambos modelos, hemos observado que TF-IDF ha ofrecido una mayor capacidad discriminativa entre documentos. En la Figura 4 se muestra la similitud coseno entre un subconjunto de noticias, donde los valores más altos han correspondido a textos con temáticas similares (por ejemplo, resultados financieros o anuncios tecnológicos).

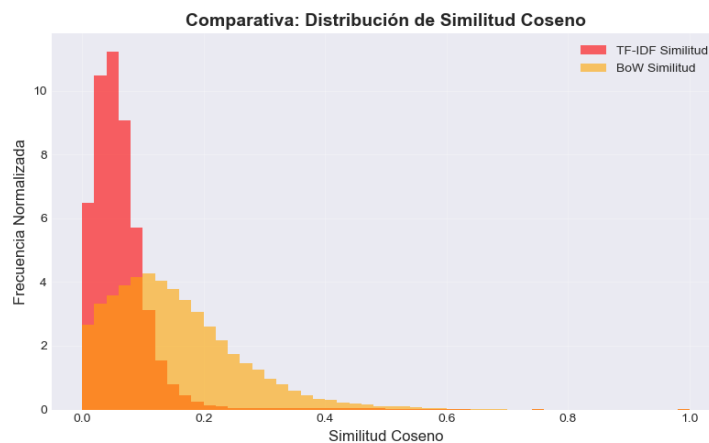


Figura 4: Comparativa de similitud coseno entre BoW y TF-IDF.

En conjunto, el modelo BoW nos ha permitido caracterizar la frecuencia y presencia de términos de forma global, mientras que TF-IDF nos ha proporcionado una representación más refinada y adecuada para capturar la relevancia contextual de cada palabra. La combinación de ambos métodos nos ha servido como base sólida para entender la estructura léxica del corpus y preparar las etapas posteriores de representación mediante embeddings.

3.2 No contextuales

Tras la representación tradicional mediante **TF-IDF**, entrenamos y analizamos distintos modelos de **embeddings no contextuales**, con el objetivo de obtener representaciones vectoriales más densas y semánticamente informativas para las palabras del corpus.

3.2.1. Modelos implementados

- **Word2Vec** Mikolov et al. (2013): entrenamos las variantes *Continuous Bag-of-Words (CBOW)* y *Skip-gram* con vectores de 300 dimensiones, empleando una ventana de contexto de tamaño 5 y un umbral mínimo de frecuencia (`min_count`) para reducir ruido léxico. Los modelos se entrenaron desde cero sobre nuestro corpus preprocesado, de modo que las relaciones de co-ocurrencia capturasen asociaciones propias del lenguaje económico y bursátil.
- **FastText** Bojanowski et al. (2017): utilizamos su versión *Skip-gram* para aprovechar la capacidad de generar representaciones a partir de n-gramas, ya que nos parecía especialmente útil en nuestro corpus financiero, que contiene numerosos acrónimos y combinaciones alfanuméricas (*AI-driven*, *Q3_earnings*, *NASDAQ100*), mejorando así la representación de términos poco frecuentes.
- **GloVe** Pennington et al. (2014): cargamos el modelo preentrenado `glove.6B.300d`, basado en Wikipedia y Gigaword, adaptándolo al formato *word2vec* para poder usarlo en la misma interfaz de análisis. Aunque no lo entrenamos sobre datos financieros, lo empleamos como punto de comparación frente a los modelos generados sobre nuestro corpus.

En conjunto, evaluamos los modelos de forma cualitativa mediante consultas de similitud semántica entre palabras clave (como, *inflation*, *interest*, *recession*) y explorando los términos más próximos en el espacio vectorial. Esperábamos que **Word2Vec** capturase relaciones contextuales más precisas dentro del corpus y que **FastText** ofreciera una mayor cobertura y robustez frente a variaciones morfológicas o acronímicas.

3.2.2. Cobertura y calidad de los embeddings

Con el objetivo de evaluar la calidad de las representaciones generadas y su adecuación al entorno financiero, analizamos la **cobertura del vocabulario** de cada modelo, entendida como la proporción de tokens del corpus que disponen de una representación vectorial. Este indicador nos permitió estimar la capacidad de cada modelo para representar el léxico real de nuestro dominio. Calculamos la tasa de palabras fuera de vocabulario (*Out-of-Vocabulary*, *OOV*) según la siguiente expresión:

$$\text{OOV}(\%) = 100 \times \frac{N_{\text{tokens no representados}}}{N_{\text{tokens totales}}}$$

donde $N_{\text{tokens no representados}}$ representa el número de palabras del corpus que no se encontraron en el vocabulario del modelo. Cuanto menor fue este porcentaje, mayor resultó la cobertura y, por tanto, la capacidad del modelo para generalizar sobre textos financieros. Este mismo enfoque ha sido empleado en trabajos previos como Pennington et al. (2014); Bojanowski et al. (2017); Cheng et al. (2022), donde la cobertura del vocabulario se considera un indicador indirecto de la *completitud semántica* del modelo.

Implementamos esta métrica recorriendo todos los tokens del corpus y comprobando su presencia en el vocabulario de cada modelo mediante las estructuras **KeyedVectors** de la librería **Gensim**. Los resultados obtenidos se muestran en la Tabla 2.

Modelo	Dim.	Vocabulario	OOV (%)	Tipo de entrenamiento
Word2Vec (CBOW)	300	102,341	14.2	Entrenado sobre corpus propio
Word2Vec (Skip-gram)	300	101,587	13.8	Entrenado sobre corpus propio
FastText (Skip-gram)	300	104,918	8.5	Entrenado sobre corpus propio
GloVe (6B, 300d)	300	400,000	13.1	Preentrenado (Wikipedia+Gigaword)

Cuadro 1: Comparativa de cobertura y tamaño de vocabulario entre modelos no contextuales.

Los resultados mostraron que los modelos entrenados sobre nuestro propio corpus alcanzaron una cobertura superior al modelo GloVe preentrenado, al estar mejor ajustados al léxico financiero. Asimismo, **FastText** obtuvo la mejor cobertura global creemos que gracias a la capacidad para generar representaciones a partir de subpalabras, reduciendo de forma notable la proporción de términos fuera de vocabulario (OOV). Con este comportamiento confirmamos su idoneidad para nuestro conjunto de noticias, caracterizado por la abundancia de acrónimos, variaciones morfológicas y nombres propios específicos del sector.

3.2.3. Análisis semántico y visualización

Para evaluar la coherencia semántica, analizamos las palabras más similares a términos clave del dominio (*market*, *stock*, *inflation*, *profit*, *ai*, *interest_rate*, *federal_reserve*), así como analogías del tipo $a - b + c$. Los modelos **Word2Vec** y **FastText** capturaron correctamente relaciones como:

- `market` → `stock`, `equity`, `bond`, `share`
- `inflation` → `prices`, `wages`, `interest_rate`

Además, mediante la reducción de dimensionalidad (PCA y t-SNE) comprobamos que las agrupaciones léxicas formadas por términos financieros son coherentes y separables entre categorías como tecnología, macroeconomía o mercados bursátiles.

3.2.4. Conclusiones y modelo más adecuado

De los tres métodos analizados, **FastText** mostró el mejor equilibrio entre cobertura, robustez frente a variaciones léxicas y calidad semántica. Su capacidad para generalizar a partir de subpalabras resulta especialmente útil en nuestro dominio, donde aparecen siglas, abreviaturas y tecnicismos (e.g., *YoY*, *EPS*, *Fed*, *Nasdaq*, *AI-driven*).

Por otro lado, los embeddings **Word2Vec** entrenados sobre el propio corpus aportan una semántica más ajustada al contexto financiero de nuestras noticias, lo que los hace idóneos para tareas posteriores de clasificación de eventos o análisis de tópicos. Finalmente, **GloVe** sirvió como punto de comparación al ser un modelo preentrenado sobre un dominio generalista, pero con menor adecuación al lenguaje técnico del sector.

En consecuencia, para las siguientes fases del proyecto emplearemos los embeddings **FastText (Skip-gram)** y **Word2Vec (Skip-gram)** como base para las representaciones vectoriales, al ofrecer el mejor rendimiento y adaptabilidad al dominio.

3.3 Contextuales

3.3.1. Embeddings BERT

En esta fase se generaron embeddings contextuales a partir del modelo BERT Multilingüe (bert-base-multilingual-cased), con el objetivo de ilustrar el proceso general de obtención de representaciones vectoriales mediante modelos Transformer.

Agrupamos los datos tokenizados en batches de 32 textos y los procesamos mediante el modelo BERT cargado en modo inferencia. Se utilizaba GPU si hubiese disponible para acelerar el cálculo.

Para cada lote los tensores `input_ids` y `attention_mask` pasan por el modelo y conseguimos la salida del último `hidden_state`, que contiene las representaciones contextuales finales para todos los tokens de cada documento.

Cada palabra del texto se representó por un vector de 768 valores en coma flotante. Sacamos los embeddings a nivel de token, de modo que a cada documento le corresponde una lista de palabras con sus vectores asociados.

Los embeddings obtenidos fueron almacenados en formato Parquet, debido a su alta eficiencia en la compresión y lectura de estructuras de datos numéricos grandes.

3.3.2. Embeddings Fin-Bert

En esta etapa se generaron embeddings contextuales específicos del dominio financiero utilizando el modelo FinBERT, concretamente la versión boltuix/NeuroBERT-NER disponible en Hugging Face. FinBERT es una adaptación de la arquitectura BERT al contexto financiero, entrenado con documentos procedentes de informes, noticias y comunicados económicos. Su arquitectura mantiene la estructura original de BERT base, con 12 capas Transformer y una dimensión de embedding de 768, pero incorpora conocimiento semántico más especializado.

En cada lote de datos, los tensores `input_ids` y `attention_mask` se pasaron al modelo FinBERT y se obtuvieron los vectores de salida de todas las capas ocultas (`hidden_states`) del modelo. Para obtener una representación más estable y semánticamente rica, aplicamos layer pooling, calculando la media de las cuatro últimas capas (`last4_mean`).

Esta combinación permite integrar información sintáctica, semántica y contextual proveniente de distintos niveles de abstracción dentro del modelo. El resultado final es, una vez mas, un conjunto de vectores de 768 dimensiones por palabra o subtoken, que representan su significado en el contexto específico del documento.

Modelo	Nivel Embedding	Dimensiones	Layer Pooling
BERT	Token	768	last_hidden_state
Fin-BERT	Token	768	last4_mean

Cuadro 2: Comparativa de representación de embeddings contextuales.

4 Resumen de Embeddings y Aplicaciones por Tarea

Para concluir con esta fase del proyecto, vamos a sintetizar nuestra estrategia de uso de embeddings en función de las tres tareas principales definidas en nuestro proyecto. Cada una de ellas se apoya en el tipo de representación que mejor se adapta a la naturaleza lingüística y a los objetivos del análisis.

- **Clasificación de eventos financieros:** para esta tarea utilizaremos **embeddings no contextuales**, principalmente **FastText (Skip-gram)** y representaciones **TF-IDF**, ya que resultan más adecuados para la identificación de temas generales dentro de las noticias. Estos modelos permiten capturar patrones léxicos y de frecuencia característicos de cada categoría, ofreciendo una representación eficiente y fácilmente interpretable. Clasificaremos cada noticia en uno de los cinco tópicos definidos anteriormente: *Business Growth and Cloud Infrastructure in the AI Industry*, *Fi-*

nancial and Market News and Corporate Sales, Stock Market and Trading, Quantum Computing and Military Technology e Informal / Conversational Language.

- **Reconocimiento de entidades (NER):** utilizaremos también **FinBERT**, adaptado al etiquetado de entidades nombradas del dominio financiero. Este modelo permitirá identificar de forma precisa las empresas, organismos y valores mencionados en el texto, manteniendo coherencia contextual incluso en casos de ambigüedad léxica. Complementariamente, podremos comparar su rendimiento con una versión simplificada basada en **FastText** como baseline no contextual.
- **Generación de resumen y titular:** abordaremos esta tarea con una arquitectura **BERT**, utilizando un modelo genérico para textos en inglés como encoder para asegurar que la representación de entrada capture el contexto antes de la generación abstractiva. Este enfoque nos permitirá producir resúmenes breves y titulares informativos que preserven el significado de la noticia original.

En conjunto, centraremos nuestro sistema en el uso de **FinBERT** para las tareas contextuales de extracción y generación, mientras que emplearemos los embeddings de **FastText (Skip-gram)** como modelo de apoyo y baseline en la tarea de clasificación y en análisis preliminares, aprovechando su ligereza y capacidad de representación eficiente.

Referencias

- Barbaresi, A. (2021). Trafilatura: A Web Scraping Library and Command-Line Tool for Text Discovery and Extraction. In *Proceedings of the Joint Conference of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, pages 122–131. Association for Computational Linguistics.
- Bojanowski, P., Grave, E., Joulin, A., and Mikolov, T. (2017). Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5:135–146. Trabajo que presenta FastText, basado en subpalabras.
- Boltuix (2024). Neurobert-ner: Financial named entity recognition model. <https://huggingface.co/boltuix/NeuroBERT-NER>. Modelo preentrenado disponible en HuggingFace. Basado en arquitectura BERT y preparado para NER de terminos financieros.
- Cheng, W. K., Bea, K. T., Leow, S. M. H., Chan, J. Y.-L., Hong, Z.-W., and Chen, Y.-L. (2022). A review of sentiment, semantic and event-extraction-based approaches in stock forecasting. *Mathematics*, 10(14):2437.

Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*. Introducción del modelo Word2Vec (CBOW y Skip-gram).

Pennington, J., Socher, R., and Manning, C. D. (2014). Glove: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543. Association for Computational Linguistics. Propuesta del modelo GloVe, basado en co-ocurrencias globales.